

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-a71a39461a0b77f3e.jsonl`  
- SHA-256: `c6e26a2c5fefcb5aeafac1a6745ff22adb6a4cde84339f21d7cd2065dc22707c`  
- Source modified: `2026-07-06T19:19:02+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_e1b943dd-dac`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T19:18:19.573Z)

Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR (branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F design doc for a four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content safety / Tier 4 deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up issues F1-F7.
No code changes. Read the ADR:
git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the codebase and the other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem, Source, Policy),
src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate should_ingest_source, delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler: watched gate -> dedup -> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track C (delivery_attempts) just merged.

Ground rules:

- READ actual files, cite ADR section / file:line. You MAY run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does FilterConfig ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies, not merge-blockers.
 - Severity: high (a claim that is false, or a design gap that would break an F-phase / cause data loss), medium (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example, minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for implementation.

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).
FINDING (design): F3 'skip non-allowlisted sources' duplicates Track E's watched gate that already runs before the pipeline
severity low | section Decision ? Security stages (Tier 3) item 1; phasing F3; References (ADR 0009 generalization)
summary: Source allowlisting (reject items from sources not in an allowlist) substantially overlaps the just-merged sources.watched opt-in gate (should_ingest_source), which already blocks ingestion from non-opted-in sources before dedup and before the pipeline.
why_it_matters: The ADR says topics.allowlist 'generalizes sources.watched,' but watched is a pre-ingestion gate and the Tier-3 allowlist is a post-ingestion pipeline stage; an unwatched source never reaches the pipeline at all, so a per-policy allowlist can only ever further restrict the already-watched set. The relationship (is watched the coarse gate and allowlist the per-policy refinement?) should be stated so F3 doesn't reimplement Track E or create a confusing second source-gating mechanism.
evidence: db.py should_ingest_source + Source.watched (models.py) gate ingestion;
telethon_client.py returns early if not should_ingest_source before dedup/filter. ADR References line: 'topics.allowlist concept generalizes sources.watched.'
rec: Clarify in F3 that watched (Track E) is the ingestion gate and the Tier-3 allowlist is a narrower per-policy refinement over already-watched sources ? or fold allowlist into routing_rules ? so the two source-gates don't conflict or duplicate.

2. assistant (2026-07-06T19:18:21.900Z)

I'll verify this finding by reading the ADR and the relevant code.

3. user (2026-07-06T19:18:23.687Z)

ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)

Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config\_json`). The filter is **flat**: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a **policy pipeline** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

What we have today (post-Track-E)

Concept	Current state
FilterConfig / Policy	One JSON blob per account. Flat duration/resolution/size/exclude rules. AND semantics.
FilterEngine	Evaluates all rules; returns FilterResult(passed, reasons).
policies table	One row per account (is_default=1). config_json is the FilterConfig blob.

| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffprobe were deferred to v2 in the original plan. |

What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

Decision

Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
...
????????????????????????????????????????????????????????????
?          Policy Pipeline                                     ?
?                                                                 ?
? Tier 1 ? Metadata gates  (always runs, zero I/O)           ?
?   duration, resolution, size, codec, mime, GIF/round       ?
?   ? extends today's FilterEngine                           ?
?                                                                 ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)         ?
?   source allowlist, keyword match, channel topic tags      ?
?   ? user-expressed interests filter the stream             ?
?                                                                 ?
? Tier 3 ? Content safety  (opt-in, metadata + light I/O)    ?
?   URL/domain reputation, file metadata sanity,             ?
?   caption text scanning                                    ?
?   ? cheap checks; no download                              ?
?                                                                 ?
? Tier 4 ? Deep inspection (opt-in, heavy I/O)               ?
?   ffprobe, perceptual hash, downloaded content scan        ?
?   ? gated behind explicit user opt-in + budget             ?
????????????????????????????????????????????????????????????
...
```

Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```
```python
@dataclass
class PolicyContext:
 """Immutable snapshot of what the stage can inspect."""
 content_item: ContentItem # from ADR 0011
 source: Source # the source it came from
 policy: Policy # the owning policy config

@dataclass
class StageResult:
 passed: bool
 reason: str # human-readable explanation
 stage_name: str # e.g. "duration-gate", "url-reputation"
 metadata: dict[str, object] = field(default_factory=dict) # for explainability UI

class PolicyStage(Protocol):
 """A single gate in the pipeline. Stateless; may be async for I/O stages."""
```

```
... async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult`` list is the **explanation trace** ? every decision is auditable.

## Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json`` under a `topics`` key:

```
```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any" // "any" = match any include topic; "all" = match all
  }
}
```
```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

## Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist`` (an explicit list of trusted `provider_source_ids`).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint`` across accounts (already handled by `content_items.dedup_key``; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata`` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}``).

## Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated behind explicit opt-in** because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true`` in `config_json``) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

## Policy evaluation flow (post-dedup, pre-delivery)

```
```python
async def evaluate_pipeline(
    ctx: PolicyContext,
    stages: list[PolicyStage],
```

```

) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results # short-circuit
    return True, results
...

```

The caller (telethon_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table (for explainability ? deferred to implementation).

Schema ? minimal changes

****No schema migration**** is required for this design. The `policies` table already holds a `config\_json` JSON blob. The pipeline is a code concept:

- `policies.config\_json` gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy\_decisions` table (deferred) would store the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.

Implementation phasing (future issues)

Phase	Scope	Dependencies
F1	Policy engine prototype	PolicyStage protocol, evaluate_pipeline(), wire into live-message handler
F2	Topic preferences	topics config parsing, source title/caption keyword matching
F3	Source allowlist	allowlist config, skip non-allowlisted sources
F4	URL/domain reputation	Domain blacklist, caption link extraction
F5	Deep inspection	ffprobe/perceptual hash, download budget enforcement
F6	Policy decisions table	Audit trail storage, explainability UI
F7	Explainability endpoint	/why <content_item_id> command showing the explanation trace

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the ****design contract**** the phases code against.

Consequences

****Positive****

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

****Negative****

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict

- catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

Alternatives considered

- ****Do nothing; keep flat FilterConfig.**** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- ****Inline each stage in the handler**** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- ****Use a rules engine / DSL.**** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- ****Bundle all phases into a single PR.**** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

Open questions for the decision owner

1. ****`StageResult.metadata` shape**** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. ****Policy decisions table**** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. ****Topic matching**** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. ****Deep inspection budget**** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

References

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

4. assistant (2026-07-06T19:18:25.570Z)

Now let me examine the relevant code to verify the finding.

5. user (2026-07-06T19:18:28.351Z)

```
c4c4d18 fix(scanning): don't clobber user-controlled watched on source upsert (#41)
3bfb1da feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR
0009, Track E) (#39)
35f4f70 feat(commands): extract CommandService ? provider-independent command execution (ADR
0011, Track D) (#38)
12d21b9 feat(delivery): destinations, routing_rules, delivery_attempts (ADR 0011, Track C) (#37)
d806b46 Merge pull request #36 from avidullu/chore/format-commands
---SHOULD_INGEST---
main:src/tg_video_filter/db.py:99:      watched          INTEGER NOT NULL DEFAULT 0,
```

```

main:src/tg_video_filter/db.py:522: # 1a. Backfill sources from channels first (known title +
watched=1). Order
main:src/tg_video_filter/db.py:523: # matters: channels-first so title/watched data wins;
the videos-based
main:src/tg_video_filter/db.py:528: (account_id, provider, provider_source_id, title,
kind, watched,
main:src/tg_video_filter/db.py:539: # ? the channels-based row above survives with its
title + watched=1.
main:src/tg_video_filter/db.py:543: (account_id, provider, provider_source_id, kind,
watched, seen_first_at)
main:src/tg_video_filter/db.py:851: watched=bool(row["watched"]),
main:src/tg_video_filter/db.py:1110: ``watched`` is **user-controlled state** (toggled only
via
main:src/tg_video_filter/db.py:1111: :func:`set_source_watched` / the ``/watch`` and
``/unwatch`` commands) and is
main:src/tg_video_filter/db.py:1114: so clobbering ``watched`` here would silently un-watch a
channel the user
main:src/tg_video_filter/db.py:1115: opted into after its first ingested video (ADR 0009).
``watched`` is only set
main:src/tg_video_filter/db.py:1122: (account_id, provider, provider_source_id,
title, kind, watched,
main:src/tg_video_filter/db.py:1134: int(source.watched),
main:src/tg_video_filter/db.py:1411: watched=False,
main:src/tg_video_filter/db.py:1546:async def set_source_watched(
main:src/tg_video_filter/db.py:1547: conn: aiomysqlite.Connection, account_id: int, source_id:
int, watched: bool
main:src/tg_video_filter/db.py:1549: """Toggle the ``watched`` flag on a source (ADR 0009,
Track E)."""
main:src/tg_video_filter/db.py:1551: "UPDATE sources SET watched = ? WHERE id = ? AND
account_id = ?",
main:src/tg_video_filter/db.py:1552: (int(watched), source_id, account_id),
main:src/tg_video_filter/db.py:1558: conn: aiomysqlite.Connection, account_id: int, *,
watched_only: bool = False
main:src/tg_video_filter/db.py:1560: """List all sources for an account, optionally filtering
to watched only."""
main:src/tg_video_filter/db.py:1564: if watched_only:
main:src/tg_video_filter/db.py:1565: query += " AND watched = 1"
main:src/tg_video_filter/db.py:1566: query += " ORDER BY watched DESC, provider_source_id
ASC"
main:src/tg_video_filter/db.py:1572:async def should_ingest_source(
main:src/tg_video_filter/db.py:1577: - Absent sources row: create it lazily with
``watched=False`` for
main:src/tg_video_filter/db.py:1578: ``/channels`` discoverability; return False (not yet
watched).
main:src/tg_video_filter/db.py:1579: - Existing row: respect the ``watched`` flag.
main:src/tg_video_filter/db.py:1586: # unwatched row so /channels can list it (ADR 0009
lines 48-54).
main:src/tg_video_filter/db.py:1594: watched=False,
main:src/tg_video_filter/db.py:1598: return src.watched
main:src/tg_video_filter/db.py:1616: watched=False,
main:src/tg_video_filter/db.py:1784: "set_source_watched",
main:src/tg_video_filter/db.py:1786: "should_ingest_source",

```

6. user (2026-07-06T19:18:29.190Z)

```

main:src/tg_video_filter/telethon_client.py:184: # --- watched gate (ADR 0009, Track E)
-----
main:src/tg_video_filter/telethon_client.py:185: # Opt-in: a source is ingested only when
watched=True. An absent
main:src/tg_video_filter/telethon_client.py:187: # unwatched and lazily create a
watched=0 row for /channels
main:src/tg_video_filter/telethon_client.py:189: if not await
db.should_ingest_source(conn, user.id, channel_id):
main:src/tg_video_filter/telethon_client.py:190: return # skip unwatched / unknown
source

```

7. assistant (2026-07-06T19:18:31.352Z)

Now let me see the full order of operations in the handler and the ADR 0009 content.

8. user (2026-07-06T19:18:34.062Z)

```
channel_id = _peer_id(message.peer_id)
message_id = message.id
document_id = _extract_document_id(message)

# --- watched gate (ADR 0009, Track E) -----
# Opt-in: a source is ingested only when watched=True. An absent
# sources row means the channel has never been seen ? treat it as
# unwatched and lazily create a watched=0 row for /channels
# discoverability (ADR 0009 lines 37-44, 130-131).
if not await db.should_ingest_source(conn, user.id, channel_id):
    return # skip unwatched / unknown source

# --- three-tier dedup (cheapest first) ---

# Tier 0: same (user, channel, message) instance ? already processed.
existing = await db.get_video(conn, user.id, channel_id, message_id)
if existing is not None:
    return

# Tier 1: same Telegram document.id ? forwarded/cross-posted file.
if document_id is not None:
    existing = await db.get_video_by_document_id(conn, user.id, document_id)
    if existing is not None:
        log.info(
            "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
            "original_channel=%s original_msg=%s",
            user.id,
            document_id,
            channel_id,
            message_id,
            existing.channel_id,
            existing.message_id,
        )
        return

# Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
if (
    meta.size_bytes is not None
    and meta.duration_s is not None
    and meta.height is not None
    and meta.width is not None
):
    existing = await db.get_video_by_content_fingerprint(
        conn,
        user.id,
        meta.size_bytes,
        meta.duration_s,
        meta.height,
        meta.width,
    )
    if existing is not None:
        log.info(
            "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
            "channel=%s msg=%s original_channel=%s original_msg=%s",
            user.id,
            meta.size_bytes,
            meta.duration_s,
            meta.height,
            meta.width,
            existing.channel_id,
            existing.message_id,
            existing.original_channel_id,
            existing.original_message_id,
        )
        return
```

channel_id,
message_id,

9. user (2026-07-06T19:18:34.729Z)

ADR 0009: Selective channel scanning and multiplexed routing

- **Status**: Accepted
- **Date**: 2026-07-04
- **Decision owner**: project owner

Context

Today the userbot scans **every** channel/group the account is a member of. ``manager.py`` registers ``events.NewMessage(incoming=True)`` with no ``chats=`` filter, and ``on_new_message`` processes every video regardless of source. The ``channels`` table exists but is populated lazily as a *record* of seen channels; it is not used as a filter.

The product vision is a **multiplexing video re-distribution system**:

- A small set of high-quality **source** channels (e.g. spiritual, classical music) are scanned for incoming videos.
- Matches are routed to themed **destination** broadcast channels (e.g. a Reggae channel, a Rap channel) that subscribers opt into.
- A user therefore subscribes to *outputs* by topic, not to *inputs* by source.

This requires two capabilities the current architecture does not have:

1. **Selective scanning** ? choose which source channels are watched, rather than ingesting all of them.
2. **Per-source ? per-destination routing** ? a match from source channel X is delivered to a specific destination channel Y, instead of the single global ``delivery_target`` (ADR 0005).

Decision

Add three additive, independently-shippable capabilities:

1. Selective scanning (watched-flag)

Add a ``watched`` flag to the ``channels`` table. The default is locked as **opt-in**: ``DEFAULT 0``, so a freshly-onboarded user scans nothing until they explicitly ``/watch`` a channel. This matches the multiplexing product shape (sources are a *curated* set) and avoids blindly re-ingesting every chat the account is a member of.

Gate ``on_new_message`` at the top: after computing ``channel_id``, look up the row and ``return`` early if ``watched`` is false, before any dedup or filter work.

This is a small migration plus ~5 lines in ``telethon_client.py``.

Discoverability note. With ``DEFAULT 0`` plus the current lazy population (a ``channels`` row is only inserted when a message is first seen), opt-in is necessary but not sufficient for discoverability: a channel the account has never received a message from has no row to flag. The ``/channels`` command (step 2 below) therefore upserts candidate rows from ``client.get_dialogs()`` so users can browse and watch channels they have not yet received traffic from. The flag default and the enumeration flow are orthogonal ? both are required.

2. Channel enumeration and management commands

New Saved Messages commands (and, via ADR 0008, bot commands) building on the existing `commands.py` pattern:

- `/channels` ? list the user's channels (via `client.get\_dialogs()`), already used in `backfill.py`) with their watched state.
- `/watch <#name|id>` ? set `watched=1` for a source channel.
- `/unwatch <#name|id>` ? set `watched=0`.
- `/watchlist` ? list only watched sources.

3. Per-source ? per-destination routing rules

Introduce a `routing\_rules` table (the canonical name; issue #25's `routes` refers to the same concept and should be aligned to `routing\_rules`), e.g.:

```
...
routing_rules (
    user_id          INTEGER NOT NULL REFERENCES users(id),
    source_channel_id INTEGER NOT NULL,
    dest_channel_id  INTEGER NOT NULL, -- a broadcast channel / @username
    filter_config_id INTEGER,          -- optional per-route filter override
    PRIMARY KEY (user_id, source_channel_id, dest_channel_id)
)
```

10. assistant (2026-07-06T19:18:40.118Z)

The finding is well-supported. The watched gate (`should\_ingest\_source`) runs at the top of `on\_new\_message` (telethon_client.py:189) and returns early before dedup and before any filter/pipeline work. An unwatched source never reaches the pipeline. Let me confirm the ADR's exact reference line and F3 wording, which I already have from the ADR read.

Let me verify the "generalizes sources.watched" claim precisely and check whether the ADR distinguishes pre-ingestion vs pipeline stage anywhere.

11. user (2026-07-06T19:18:42.654Z)

```
55:?    source allowlist, keyword match, channel topic tags    ?
125:1. **Source allowlist**: reject items from sources not in the `allowlist`
174:- `policies.config_json` gains optional `topics`, `allowlist`,
187:| **F3** ? Source allowlist | `allowlist` config, skip non-allowlisted sources | F1 |
256:- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept
257: generalizes `sources.watched`).
```

12. assistant (2026-07-06T19:18:53.982Z)

I have enough to reach a verdict.

The finding holds against the ADR. Key facts verified:

1. **Watched is a pre-ingestion gate, allowlist is a pipeline stage.** `should\_ingest\_source` runs at the very top of `on\_new\_message` (telethon_client.py:189), returning early before dedup and before the FilterEngine/pipeline. ADR 0009 itself says to "Gate `on\_new\_message` at the top... `return` early if `watched` is false, before any dedup or filter work." So an unwatched source never reaches the Tier-3 pipeline at all.

2. **ADR 0012's Tier 3 Source allowlist** (Decision ? Security stages, item 1, line 125) says "reject items from sources not in the `allowlist`" and F3 (line 187) says "skip non-allowlisted sources" ? a post-dedup, pre-delivery pipeline stage per the "Policy evaluation flow (post-dedup, pre-delivery)" section.

3. **The References line** (256-257) claims "the `topics.allowlist` concept generalizes `sources.watched`" ? but the ADR never reconciles the two source-gating mechanisms. It never states that watched is the coarse ingestion gate and the allowlist is a per-policy refinement, nor does it note that the allowlist can only further restrict the already-watched set.

The overlap is real: both gate by source. It's genuinely a gap that F3 could reimplement watched or create a confusing second source gate. But it is a clarification gap, not a false claim or data-loss risk ? the two mechanisms don't actually conflict (allowlist can only narrow), and a design doc reasonably defers such detail to the F3 phase. This calibrates to low/medium; the reported "low" is appropriate, arguably medium since it's an unspecified interaction between a just-merged track and a planned phase. I'll set corrected severity to low, consistent with the report and the fact that the mechanisms are non-conflicting.

13. user (2026-07-06T19:19:02.197Z)

Structured output provided successfully